

# Shiv Nadar University Chennai

## CS3807 – Deep Learning Laboratory

### Experiment 3

## Implementation of Convolutional Neural Networks (CNNs) for Image Classification

|                     |                                               |             |
|---------------------|-----------------------------------------------|-------------|
| Degree & Branch     | B.Tech Artificial Intelligence & Data Science | Semester V  |
| Subject Code & Name | CS3807 – Deep Learning Laboratory             | AY: 2026–27 |

## 1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

## 2. Learning Outcomes

Students will be able to

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

## 3. Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

where  $X$  is the input image,  $K$  is the kernel (filter), and  $Y$  is the feature map.

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where  $N$  is the input size,  $F$  is the filter size,  $P$  is the padding, and  $S$  is the stride.

### 3.1 Common Convolution Kernels

A **kernel** (or **filter**) is a small matrix that slides over an input image to extract visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise

with the corresponding image pixels, and the results are summed to produce a value in the output feature map.

### 1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

### 2. Sobel-X Kernel (Vertical Edge Detection)

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

The Sobel-X kernel detects vertical edges by measuring intensity changes along the horizontal direction.

### 3. Sobel-Y Kernel (Horizontal Edge Detection)

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

The Sobel-Y kernel detects horizontal edges by measuring intensity changes along the vertical direction.

### 4. Laplacian Kernel

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

### 5. Sharpening Kernel

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

This kernel enhances fine details by emphasizing high-frequency components.

## 6. Box Blur Kernel

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

The Box Blur kernel smooths an image by replacing each pixel with the average of neighboring pixels.

## 7. Gaussian Blur Kernel

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

## 8. Emboss Kernel

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

## 9. Outline Detection Kernel

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

## 10. Motion Blur Kernel

$$\frac{1}{5} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

This kernel simulates motion blur along the diagonal direction.

## Summary of Common Kernels

| Kernel        | Size         | Purpose                        | Applications        |
|---------------|--------------|--------------------------------|---------------------|
| Identity      | $3 \times 3$ | Preserve original image        | Baseline comparison |
| Sobel-X       | $3 \times 3$ | Detect vertical edges          | Object detection    |
| Sobel-Y       | $3 \times 3$ | Detect horizontal edges        | Boundary detection  |
| Laplacian     | $3 \times 3$ | Detect edges in all directions | Medical imaging     |
| Sharpen       | $3 \times 3$ | Enhance details                | Image enhancement   |
| Box Blur      | $3 \times 3$ | Smooth image                   | Noise removal       |
| Gaussian Blur | $3 \times 3$ | Reduce noise                   | Computer vision     |
| Emboss        | $3 \times 3$ | 3D effect                      | Artistic filtering  |
| Outline       | $3 \times 3$ | Boundary extraction            | Segmentation        |
| Motion Blur   | $5 \times 5$ | Simulate motion                | Video processing    |

## 4. Numerical Example 1: Convolution

Consider

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

The four valid convolution positions give

$$6, \quad 8, \quad 12, \quad 14$$

and hence the feature map is

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}.$$

## 5. Numerical Example 2: Max Pooling

For the feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

a  $2 \times 2$  max-pooling operation gives

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}.$$

## 6. Numerical Example 3: Parameter Calculation

For an input image of  $32 \times 32 \times 3$ , 16 filters, and a  $3 \times 3$  kernel:

$$(3 \times 3 \times 3 + 1) \times 16 = 448.$$

Therefore, the convolution layer contains **448 trainable parameters**.

## 7. Dataset

The experiment uses the **CIFAR-10** dataset.

- Training Images: 50,000
- Testing Images: 10,000
- Classes: 10
- Image Size:  $32 \times 32 \times 3$

The ten classes used in the notebook are:

airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck

## 8. Experimental Procedure

### Task 1: Load and Inspect CIFAR-10

The CIFAR-10 dataset is loaded using TensorFlow/Keras. Ten sample training images are displayed, the dataset dimensions are printed, and class distributions are plotted for both training and test sets.



Figure 1: Sample CIFAR-10 training images.

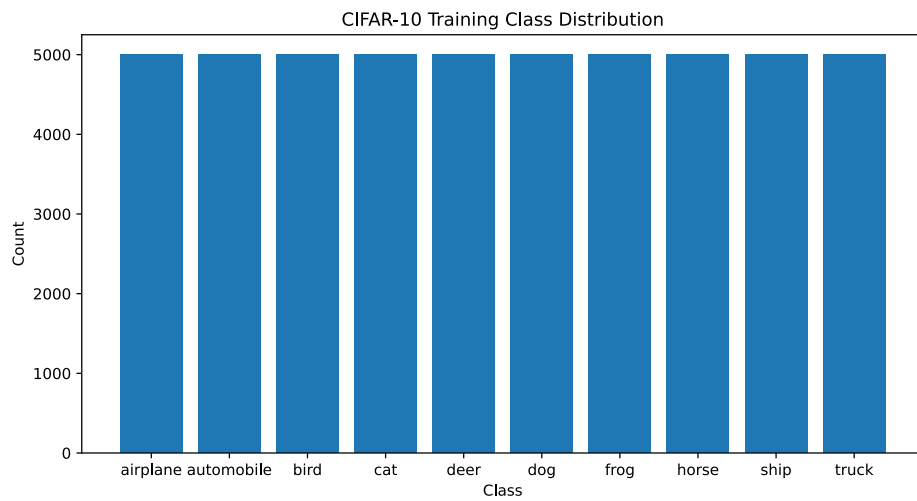


Figure 2: CIFAR-10 training class distribution.

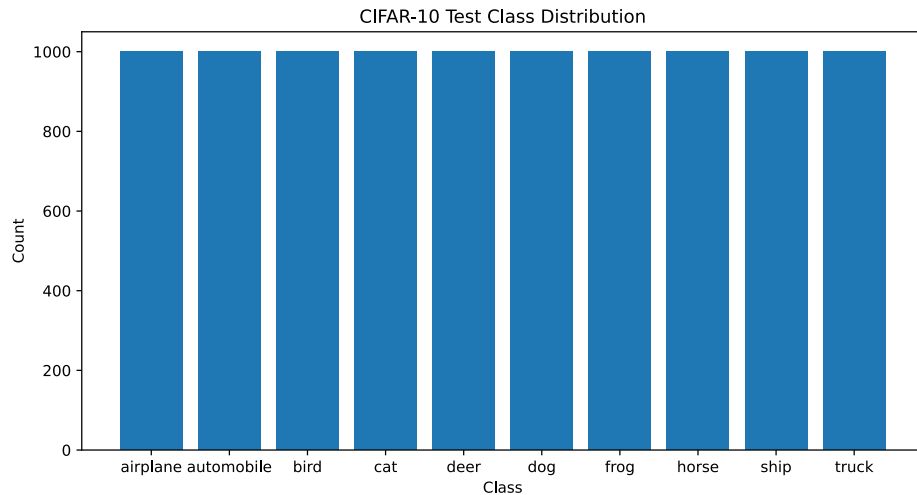


Figure 3: CIFAR-10 test class distribution.

**Inference:** The sample-image plot provides a visual view of the ten CIFAR-10 categories. The class-distribution plots show the number of examples available for each class in the training and test sets.

## Task 2: Implement a Convolution Layer

The notebook compares feature-map sizes for  $3 \times 3$ ,  $5 \times 5$ , and  $7 \times 7$  convolution kernels using stride 1 and `valid` padding for an input of  $32 \times 32 \times 3$ .

| Kernel       | Stride | Feature Map Size |
|--------------|--------|------------------|
| $3 \times 3$ | 1      | $30 \times 30$   |
| $5 \times 5$ | 1      | $28 \times 28$   |
| $7 \times 7$ | 1      | $26 \times 26$   |

**Inference:** With `valid` padding and stride 1, increasing the kernel size reduces the spatial dimensions of the output feature map.

## Task 3: Study Hyperparameters – Stride and Padding

A  $3 \times 3$  kernel is used to compare different stride and padding configurations.

| Configuration                | Input          | Output Spatial Size |
|------------------------------|----------------|---------------------|
| Stride 1, <code>valid</code> | $32 \times 32$ | $30 \times 30$      |
| Stride 2, <code>valid</code> | $32 \times 32$ | $15 \times 15$      |
| Stride 1, <code>same</code>  | $32 \times 32$ | $32 \times 32$      |
| Stride 1, <code>valid</code> | $32 \times 32$ | $30 \times 30$      |

**Inference:** Increasing the stride reduces the spatial resolution more aggressively. With `same` padding and stride 1, the spatial dimensions are preserved.

#### Task 4: Visualize Feature Maps

A single convolutional layer with 32 filters of size  $3 \times 3$  is applied to a normalized CIFAR-10 image. The first eight feature maps are displayed.

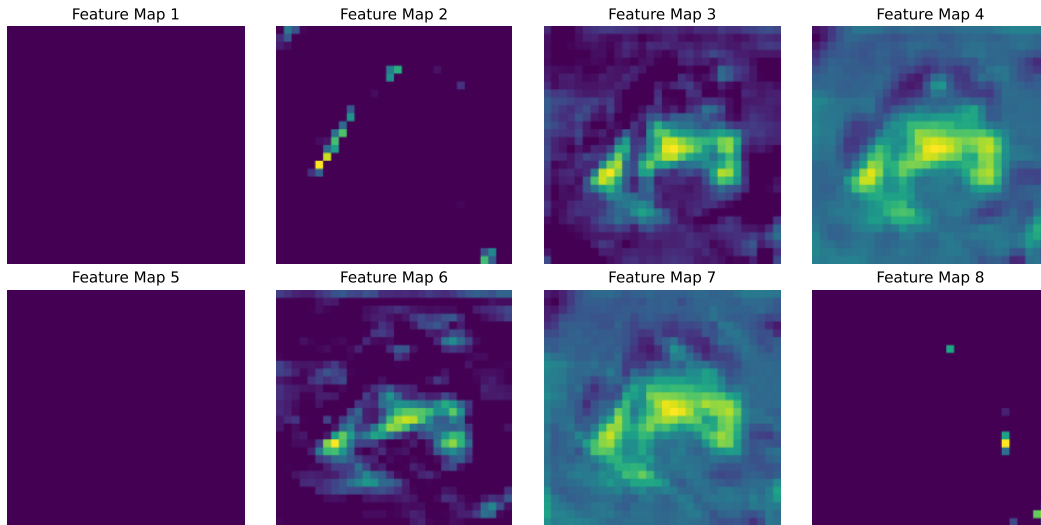


Figure 4: Feature maps produced by the first convolutional layer.

**Inference:** Each feature map represents the response of a learned filter to the sample image. Early convolutional filters can respond to structures such as edges, textures, and patterns.

#### Task 5: Compare Max Pooling and Average Pooling

The notebook uses an example input shape of  $30 \times 30 \times 32$  and a  $2 \times 2$  pooling window.

| Pooling Type    | Pool Size    | Output Spatial Size |
|-----------------|--------------|---------------------|
| Max Pooling     | $2 \times 2$ | $15 \times 15$      |
| Average Pooling | $2 \times 2$ | $15 \times 15$      |

The full CNN is subsequently trained with both pooling types for accuracy comparison.

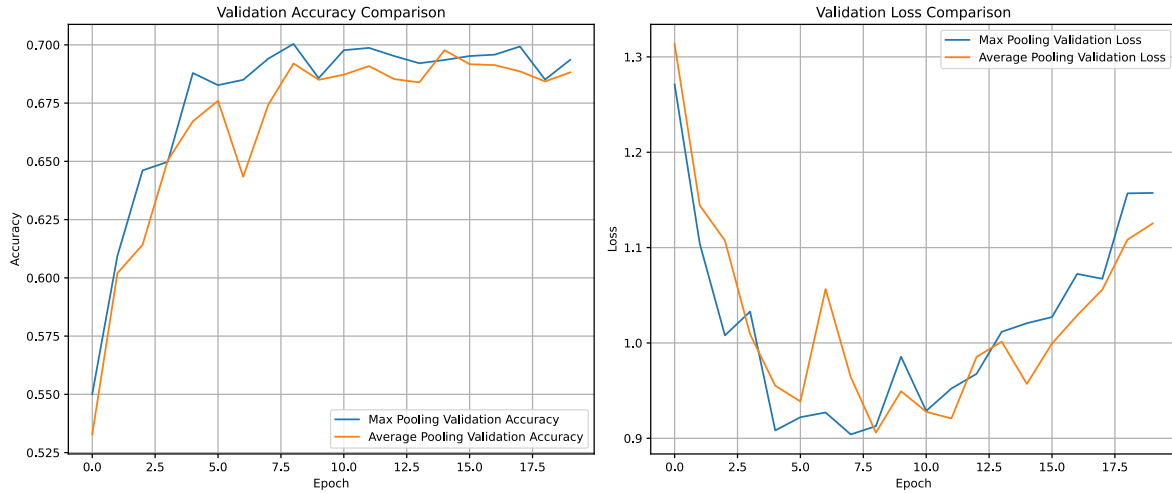


Figure 5: Validation accuracy and validation loss comparison for max pooling and average pooling.

**Inference:** Both pooling operations reduce the spatial dimensions from  $30 \times 30$  to  $15 \times 15$ . The comparison plot shows how the two choices affect validation accuracy and validation loss during training.

## Task 6: Construct and Train the CNN

The CNN implemented in the notebook is:

Input  $\rightarrow$  Conv2D( $32, 3 \times 3$ )  $\rightarrow$  MaxPool( $2 \times 2$ )  $\rightarrow$  Conv2D( $64, 3 \times 3$ )  $\rightarrow$  MaxPool( $2 \times 2$ )  $\rightarrow$  Flatten  $\rightarrow$  Dense( $64$ )  $\rightarrow$  Dense( $10$ )

The input pixels are scaled to the range  $[0, 1]$ . The model is compiled with the Adam optimizer and sparse categorical cross-entropy loss and is trained for 20 epochs with a batch size of 32.

For the stated architecture, the trainable parameter count is:

$$\begin{aligned}
 \text{Conv1} &= (3 \times 3 \times 3 + 1) \times 32 = 896, \\
 \text{Conv2} &= (3 \times 3 \times 32 + 1) \times 64 = 18,496, \\
 \text{Dense} &= (6 \times 6 \times 64) \times 64 + 64 = 147,520, \\
 \text{Output} &= (64 + 1) \times 10 = 650.
 \end{aligned}$$

Thus,

$$\boxed{\text{Total trainable parameters} = 167,562.}$$

## Task 7: Evaluate the Model

The trained model is evaluated on the scaled test set. The notebook computes test loss and accuracy, generates a classification report containing precision, recall, and F1-score, and creates a confusion matrix.

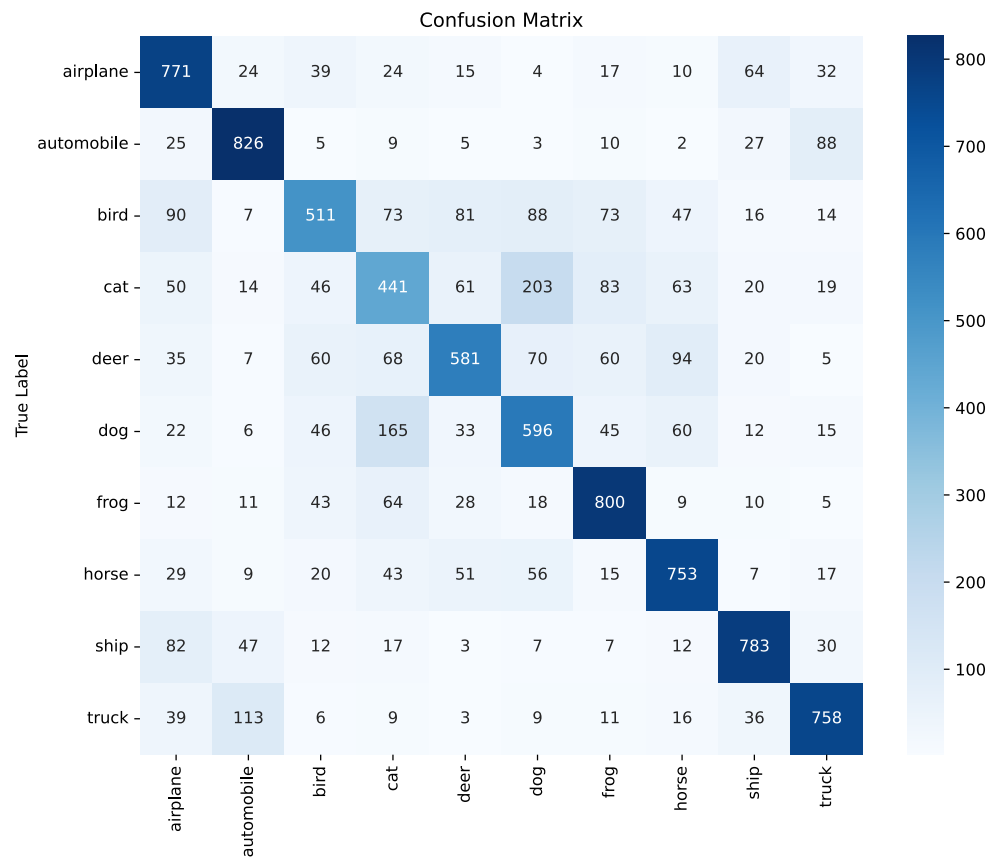


Figure 6: Confusion matrix for the CIFAR-10 test predictions.

**Inference:** The diagonal entries of the confusion matrix represent correctly classified images, while off-diagonal entries show class confusions. The classification report provides class-wise precision, recall, and F1-score.

## Task 8: Training and Validation Accuracy

The notebook records training and validation accuracy over 20 epochs.

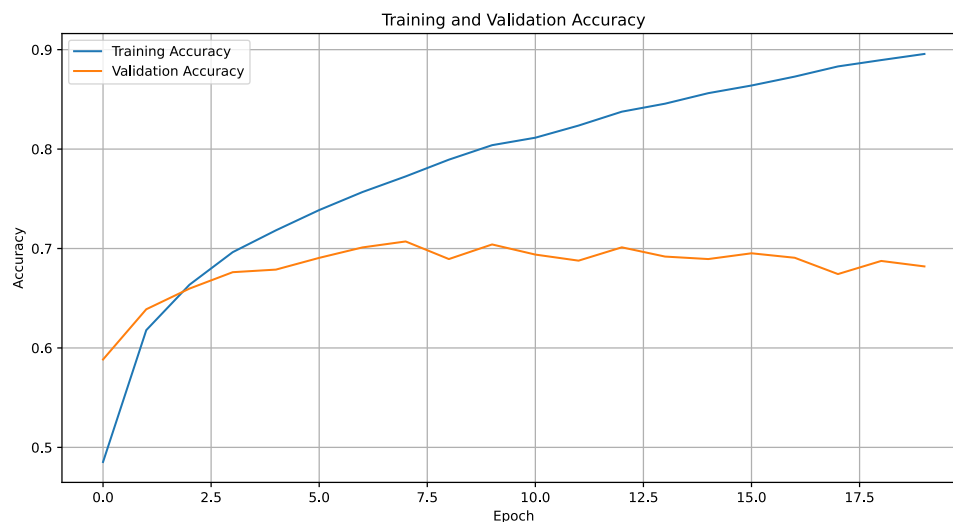


Figure 7: Training and validation accuracy over epochs.

**Inference:** The curves show the learning behavior of the model across epochs. The gap between training and validation accuracy can be used to assess generalization and possible overfitting.

## Task 9: Training and Validation Loss

The notebook records training and validation loss over 20 epochs.

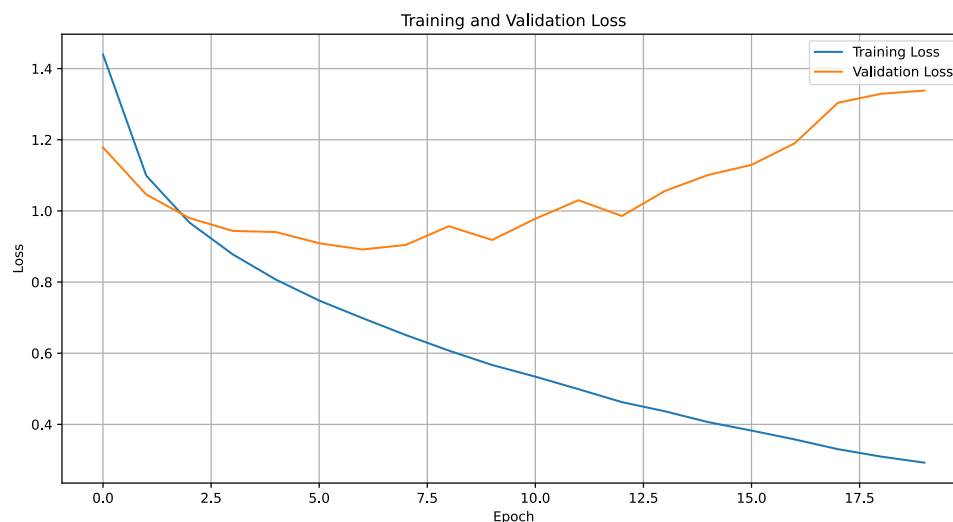


Figure 8: Training and validation loss over epochs.

**Inference:** A decreasing loss generally indicates that the model is learning from the training data. A rise in validation loss while training loss continues to fall can indicate overfitting.

## 9. Additional Exercises

### 1. Output Size for a $64 \times 64$ Image

For a  $64 \times 64$  image,  $5 \times 5$  kernel, stride 2, and explicit padding 2:

$$\frac{64 + 2(2) - 5}{2} + 1 = \frac{63}{2} + 1 = 32.5.$$

Using the integer/floor output convention used for convolution output dimensions, the output is

$$\boxed{32 \times 32}.$$

### 2. Trainable Parameters for 64 Filters

For 64 filters of size  $3 \times 3$  with an RGB input:

$$(3 \times 3 \times 3 + 1) \times 64 = 28 \times 64 = \boxed{1,792}.$$

### 3. ReLU and Sigmoid

**ReLU:**

$$f(x) = \max(0, x).$$

It is computationally efficient, promotes sparsity, and helps mitigate the vanishing-gradient problem. A disadvantage is the possibility of “dying ReLU” neurons.

**Sigmoid:**

$$f(x) = \frac{1}{1 + e^{-x}}.$$

It maps values to the range  $(0, 1)$  and is useful when probability interpretation is required, particularly for binary classification outputs. It can suffer from vanishing gradients for very large or very small inputs and is not zero-centered.

**Comparison:** ReLU is generally preferred in hidden layers, while sigmoid is commonly used for binary classification outputs.

### 4. Replace Max Pooling with Average Pooling

An otherwise identical CNN is constructed by replacing both `MaxPool2D` layers with `AveragePooling2D`. The model is trained for 20 epochs with batch size 32 and its test accuracy is compared with the max-pooling model.

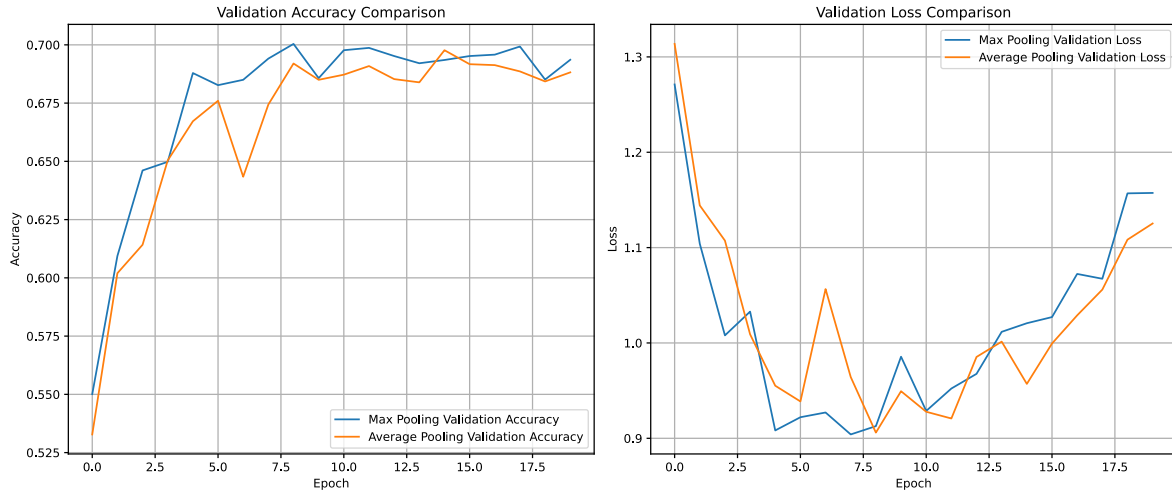


Figure 9: Comparison of validation accuracy and validation loss for max pooling and average pooling.

**Inference:** The plot provides the experimental comparison required to determine which pooling operation gives better validation behavior for this implementation.

## 5. Increase the Number of Convolution Filters

The notebook compares:

- Fewer-filter model: 16 filters in the first convolution layer and 32 in the second.
- Original model: 32 filters in the first convolution layer and 64 in the second.

The models are trained for 20 epochs with batch size 32 and their validation curves and training times are compared.

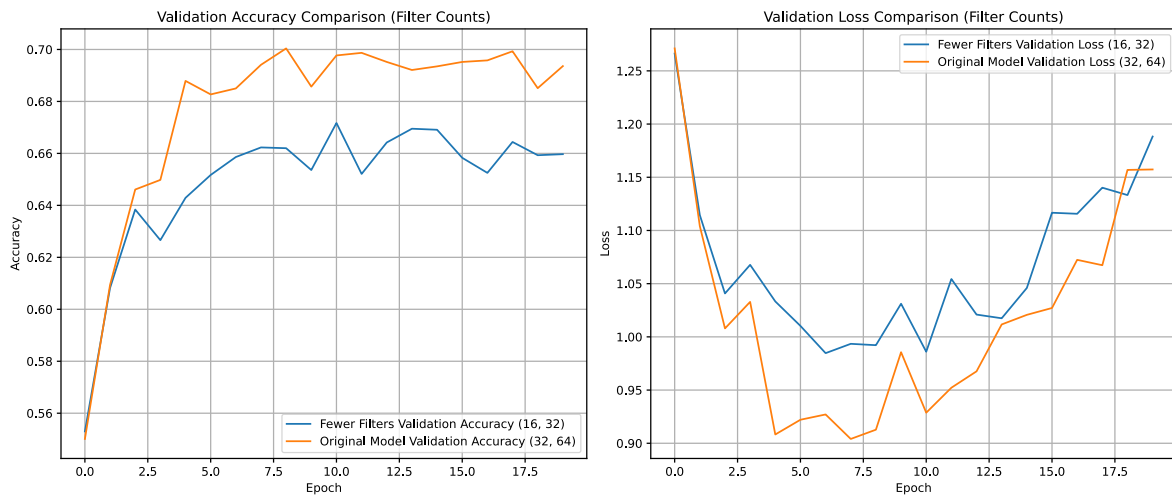


Figure 10: Validation accuracy and validation loss comparison for different convolution filter counts.

For the fewer-filter architecture, the parameter count is:

$$\begin{aligned}\text{Conv1} &= (3 \times 3 \times 3 + 1) \times 16 = 448, \\ \text{Conv2} &= (3 \times 3 \times 16 + 1) \times 32 = 4,640, \\ \text{Dense} &= (6 \times 6 \times 32) \times 64 + 64 = 73,792, \\ \text{Output} &= 650.\end{aligned}$$

Hence,

$$\boxed{\text{Total} = 79,530}.$$

**Inference:** Increasing the number of convolution filters increases representational capacity and the number of trainable parameters. The actual accuracy and training-time difference should be taken from the generated comparison plot and notebook execution output rather than assumed.

## 10. Results

The following quantities are produced by the notebook execution:

| Metric                        | Value / Source  |
|-------------------------------|-----------------|
| Training Accuracy             | 0.86            |
| Validation/Test Accuracy      | 0.6878 and 0.69 |
| Precision                     | 0.69            |
| Recall                        | 0.687           |
| F1-score                      | 0.687           |
| Original Model Parameters     | 167,562         |
| Fewer-Filter Model Parameters | 79,530          |

## 11. Discussion

### 1. Why is convolution preferred over fully connected layers for images?

It uses local receptive field and also trains with fewer number of paramters than fully connected layers.

### 2. How does stride affect the feature map size?

Strides defines how much the kernel moves which essentially gives the size of feature map. Shorter strides is best for small images to extract maximum information. Whereas for larger images small strides increases complexity so in that cases a larger stride is preferred.

### 3. What is the role of padding?

It is an extra layer of generally zeroes added around the image as the corners of the image usually contributes to lesser number in the feature map so using this increases the contribution of the corners and hence giving it more priority than before.

### 4. Why is pooling used?

It is used to reduce the computation complexity. It reduces number of operation by effectively compressing the size without losing much information of the image.

### 5. How do feature maps represent image characteristics?

The feature map contains the representation of filters at different locations of the image. Thus showing different characterisitcs like the edge, the centre well which helps in classification.

## 6. Why do CNNs require fewer parameters than MLPs?

CNNs usually require less parameters than MLPs as this has an advantage of weight sharing throughout the process which reduces the number of parameters required at step as in an MLP the number would be multiplied at each step.

## 12. Conclusion

CNN model using the CIFAR 10 dataset is used and multiclass classification has been done. The main objectives of learning convolution, testing with different kernel sizes, different strides and padding are done. Feature map is visualized for seeing how the model works at each step. Pooling layer is implemented in 2 layers with both average pooling and maximum pooling and difference is compared.

## 13. Source Code

<https://github.com/ssukesh182/DeeplearningLab>

## 14. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.